

# Builder Pattern

빌더 패턴(Builder Pattern)은 **복잡한 객체 생성 과정과 표현 방법을 분리하여 다양한 구성의 인스턴스를 만드는 생성 패턴**이다. 생성자에 들어갈 매개 변수를 메서드로 하나하나 받아들이고 마지막에 통합 빌드해서 객체를 생성하는 방식이다.

이해하기 쉬운 사례로 수제 햄버거를 들 수 있다. 수제 햄버거를 주문할 때 빵이나 패티 등 속재료들은 주문하는 사람이 마음대로 결정된다. 어느 사람은 치즈를 빼달라고 할 수 있고 어느 사람은 토마토를 빼달라고 할 수 있다. 이처럼 선택적 속재료들을 보다 유연하게 받아 다양한 타입의 인스턴스를 생성할 수 있다.



별도의 Builder 클래스를 만들어 메소드를 통해 step-by-step 으로 값을 입력받은 후에 최종적으로 build() 메소드로 하나의 인스턴스를 생성하여 리턴하는 패턴이다.

빌더 패턴을 이용하면 **생성자 오버로딩 열거를 하지 않아도 되며, 데이터의 순서에 상관없이 객체를 만들어내 생성자 인자 순서를 파악할 필요도 없고 잘못된 값을 넣는 실수도 하지 않게 된다.**

```
class Student {  
    private int id;  
    private String name = "아무개";  
    private String grade = "freshman";  
    private String phoneNumber = "010-0000-0000";  
  
    public Student(int id, String name, String grade, String phoneNumber) {  
        this.id = id;  
        this.name = name;  
        this.grade = grade;  
        this.phoneNumber = phoneNumber;  
    }  
  
    @Override  
    public String toString() {
```

```

        return "Student { " +
            "id='" + id + 'W' +
            ", name=" + name +
            ", grade=" + grade +
            ", phoneNumber=" + phoneNumber +
            " }";
    }
}

```

// 빌더 클래스

```

class StudentBuilder {
    private int id;
    private String name;
    private String grade;
    private String phoneNumber;

    public StudentBuilder id(int id) {
        this.id = id;
        return this;
    }
}

```

```

    public StudentBuilder name(String name) {
        this.name = name;
        return this;
    }
}

```

```

    public StudentBuilder grade(String grade) {
        this.grade = grade;
        return this;
    }
}

```

```

    public StudentBuilder phoneNumber(String phoneNumber) {
        this.phoneNumber = phoneNumber;
        return this;
    }
}

```

```

    public Student build() {
        return new Student(id, name, grade, phoneNumber); // Student 생성자 호출
    }
}

```

// 메인 클래스

```

public static void main(String[] args) {

    Student student = new StudentBuilder()
        .id(123456789)
        .name("둘리")
        .grade("Senior")
        .phoneNumber("010-5555-5555")
        .build();

    System.out.println(student);
}

```

## Lombok 의 @Builder

개발자가 좀더 편하게 빌더 패턴을 적할 수 있도록 Lombok에서는 별도의 애노테이션을 지원한다. 클래스에 @Builder 애노테이션만 붙여주면 클래스를 컴파일 할 때 자동으로 클래스 내부에 빌더 API가 만들어진다. 단, 롬복의 @Builder는 GOF의 디렉터 빌더가 아닌 심플 빌더 패턴을 다룬다.

[ 빌더 애노테이션 구현하기 ]

**@Builder** : Xxx.Builder 클래스와 이를 반환하는 builder() 메서드, 멤버 변수명의 메서드 생성

**@AllArgsConstructor(access = AccessLevel.PRIVATE)** : @Builder 애노테이션을 선언하면 전체 인자를 갖는 생성자를 자동으로 만드는데, 이를 private 생성자로 설정

**@ToString** : toString() 메서드 자동 생성

[ 예 ]

```
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.ToString;
```

**@Builder**

**@AllArgsConstructor(access = AccessLevel.PRIVATE)**

**@ToString**

```
class Person {
    private final String name;
    private final String age;
    private final String gender;
    private final String job;
    private final String birthday;
    private final String address;
}

public static void main(String[] args) {
    Person person = Person.builder()
        .name("둘리")
        .age("10")
        .gender("man")
        .job("가정부")
        .birthday("1983.04.22")
        .address("쌍문동")
        .build();
}
```

## [ 필수 파라미터 빌더 구현하기 ]

@Builder

@AllArgsConstructor(access = AccessLevel.PRIVATE)

@ToString

class Person {

private final String name;

private final String age;

private final String gender;

private final String job;

private final String birthday;

private final String address;

// 필수 파라미터 빌더 메서드 구현

public static PersonBuilder builder(String name, String age) {

// 빌더의 파라미터 검증

if(name == null || age == null)

throw new IllegalArgumentException("필수 파라미터 누락");

// 필수 파라미터를 미리 빌드한 빌더 객체를 반환 (자연 빌더 원리)

return new PersonBuilder().name(name).age(age);

}

}

public static void main(String[] args) {

Person person = Person.builder("또치", "10") // 필수 파라미터

.gender("woman") // 선택 파라미터

.job("가정부친구")

.birthday("1983.xx.xx")

.address("아프리카")

.build();

System.out.println(person);

}

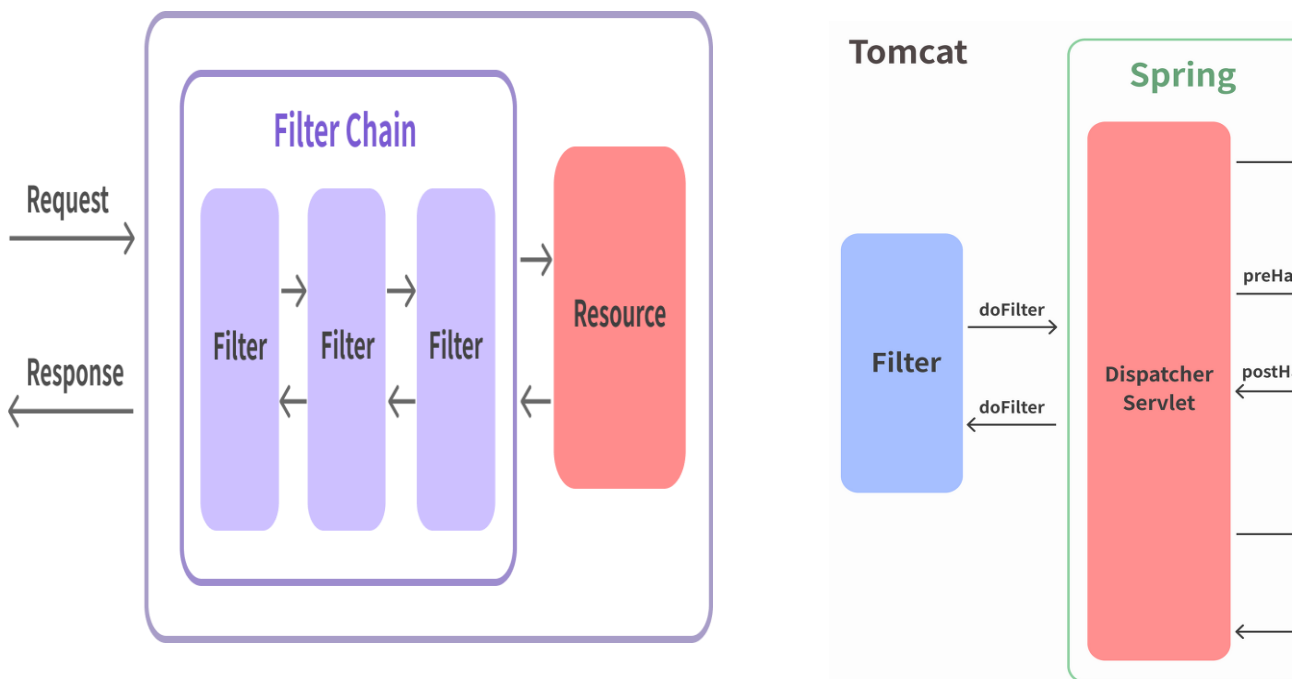
# 필터와 인터셉터

Spring은 공통적인 여러 작업을 대신 처리하며 개발시 중복된 코드를 제거할 수 있도록 많은 기능들을 지원하는데 필터(Filter)와 인터셉터(Interceptor)도 여기에 포함된다.

## [ 필터(Filter)란? ]

필터(Filter)는 J2EE 표준 스펙 기능으로 DispatcherServlet(Dispatcher Servlet)에 요청이 전달되기 전/후에 url 패턴에 맞는 모든 요청에 대해 부가작업을 처리할 수 있는 기능을 제공한다. DispatcherServlet은 스프링의 가장 앞단에 존재하는 프론트 컨트롤러이므로, 필터는 스프링 범위 밖에서 처리가 되는 것이다.

즉, 스프링 컨테이너가 아닌 톰캣과 같은 웹 컨테이너(서블릿 컨테이너)에 의해 관리가 되는 것이고(스프링 빈으로 등록은 된다), DispatcherServlet 전/후에 처리하는 것이다. 이러한 과정을 그림으로 표현하면 다음과 같다.



## [ 필터(Filter)의 메서드 ]

필터를 추가하기 위해서는 javax.servlet의 Filter 인터페이스를 구현(implements)해야 하며 이는 다음의 3가지 메서드를 가지고 있다.

```

public interface Filter {
    public default void init(FilterConfig filterConfig) throws ServletException {}
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain) throws IOException, ServletException {}
    public default void destroy() {}
}

```

#### - init 메서드

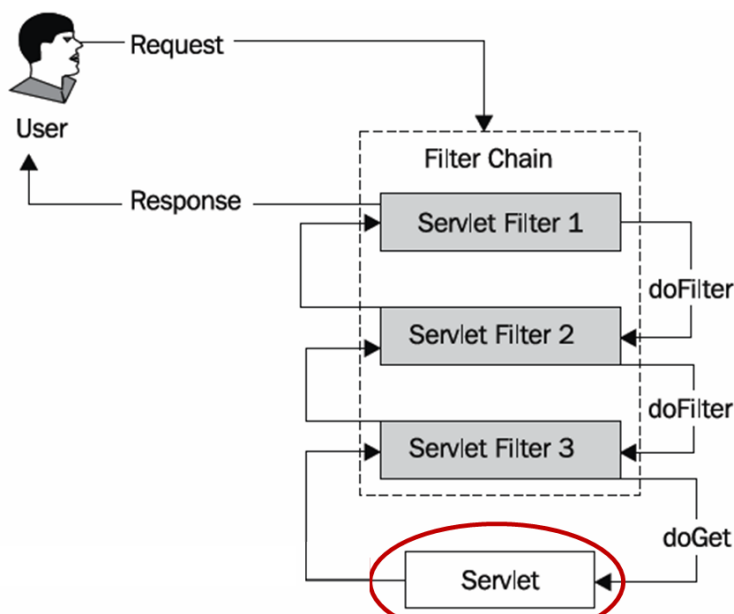
init 메서드는 필터 객체를 초기화하고 서비스에 추가하기 위한 메서드다. 웹 컨테이너가 1회 init 메서드를 호출하여 필터 객체를 초기화하면 이후의 요청들은 doFilter를 통해 처리된다.

#### - doFilter 메서드

doFilter 메서드는 url-pattern에 맞는 모든 HTTP 요청이 DispatcherServlet으로 전달되기 전에 웹 컨테이너에 의해 실행되는 메서드다. doFilter의 파라미터로는 FilterChain이 있는데, FilterChain의 doFilter 통해 다음 대상으로 요청을 전달하게 된다. chain.doFilter() 전/후에 우리가 필요한 처리 과정을 넣어줌으로써 원하는 처리를 진행할 수 있다.

#### - destroy 메서드

destroy 메서드는 필터 객체를 서비스에서 제거하고 사용하는 자원을 반환하기 위한 메서드다. 이는 웹 컨테이너에 의해 1번 호출되며 이후에는 이제 doFilter에 의해 처리되지 않는다.



## [ 필터의 등록 ]

@Component

@Slf4j

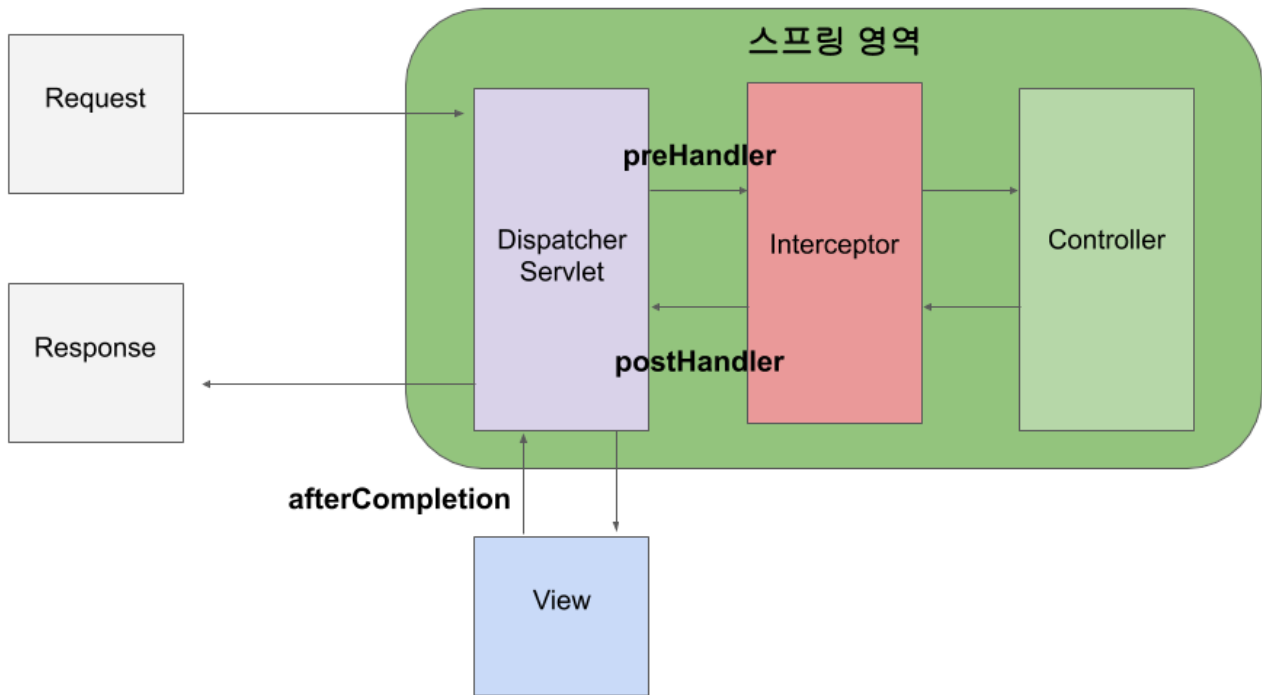
@Order(2)

```
public class TestFilter1 implements Filter {  
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain  
chain)  
        throws IOException, ServletException{  
        log.info("[필터1] 요청 자원 수행 전");  
        chain.doFilter(request, response);  
        log.info("[필터1] 요청 자원 수행 후");  
    }  
}
```

## [ 인터셉터(Interceptor)란? ]

인터셉터(Interceptor)는 J2EE 표준 스펙인 필터(Filter)와 달리 Spring이 제공하는 기술로써, DispatcherServlet(Dispatcher Servlet)이 컨트롤러를 호출하기 전과 후에 요청과 응답을 참조하거나 가공할 수 있는 기능을 제공한다. 즉, 웹 컨테이너(서블릿 컨테이너)에서 동작하는 필터와 달리 인터셉터는 스프링 컨텍스트에서 동작을 하는 것이다.

DispatcherServlet은 핸들러 매핑을 통해 적절한 컨트롤러를 찾도록 요청하는데, 그 결과로 실행 체인(HandlerExecutionChain)을 리턴해 준다. 이 실행 체인은 1개 이상의 인터셉터가 등록되어 있다면 순차적으로 인터셉터들을 거쳐 컨트롤러가 실행되도록 하고, 인터셉터가 없다면 바로 컨트롤러를 실행한다. 인터셉터는 스프링 컨테이너 내에서 동작하므로 필터를 거쳐 프론트 컨트롤러인 DispatcherServlet이 요청을 받은 **이후에 동작**하게 되는데, 이러한 호출 순서를 그림으로 표현하면 다음과 같다.



### [ 인터셉터(Interceptor)의 메서드 ]

인터셉터를 추가하려면 org.springframework.web.servlet의 HandlerInterceptor 인터페이스를 구현(implements)해야 하며, 이는 다음의 3가지 메서드를 가지고 있다.

```

public interface HandlerInterceptor {
    default boolean preHandle(HttpServletRequest request,
                               HttpServletResponse response, Object handler) throws Exception {
        return true;
    }
    default void postHandle(HttpServletRequest request, HttpServletResponse response,
                             Object handler, @Nullable ModelAndView modelAndView) throws Exception {
    }
    default void afterCompletion(HttpServletRequest request, HttpServletResponse
                                response, Object handler, @Nullable Exception ex) throws Exception {
    }
}

```



#### - preHandle() 메서드

preHandle 메서드는 컨트롤러가 호출되기 전에 실행된다. 그렇기 때문에 컨트롤러 이전에 처리해야 하는 전처리 작업이나 요청 정보를 가공하거나 추가하는 경우에 사용할 수 있다.

preHandle의 3번째 파라미터인 handler 파라미터는 핸들러 매핑이 찾아준 컨트롤러 빈에 매핑되는 HandlerMethod라는 새로운 타입의 객체로써, @RequestMapping이 붙은 메서드의 정보를 추상화한 객체이다. 또한 preHandle의 반환 타입은 boolean인데 **반환 값이 true이면 다음 단계로 진행이 되지만, false라면 작업을 중단하여 이후의 작업(다음 인터셉터 또는 컨트롤러)은 진행되지 않는다.**

#### - postHandle() 메서드

postHandle 메서드는 컨트롤러를 호출된 후에 실행된다. 그렇기 때문에 컨트롤러 이후에 처리해야 하는 후처리 작업이 있을 때 사용할 수 있다. 이 메서드에는 컨트롤러가 반환하는 ModelAndView 타입의 정보가 제공되는데, 최근에는 Json 형태로 데이터를 제공하는 RestAPI 기반의 컨트롤러(@RestController)를 만들면서 자주 사용되지는 않는다. 또한 컨트롤러 하위 계층에서 작업을 진행하다가 중간에 예외가 발생하면 postHandle은 호출되지 않는다.

#### - afterCompletion() 메서드

afterCompletion 메서드는 이름 그대로 **모든 뷰에서 최종 결과를 생성하는 일을 포함해 모든 작업이 완료된 후에 실행된다.** 요청 처리 중에 사용한 리소스를 반환할 때 사용하기에 적합하다. postHandler과 달리 컨트롤러 하위 계층에서 작업을 진행하다가 중간에 예외가 발생하더라도 afterCompletion은 반드시 호출된다.

#### [ 인터셉터의 등록 ]

```
package com.example.springedu.config;
```

```
import com.example.springedu.interceptor.TestInterceptor;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
```

```
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
```

## @Configuration

```
public class WebMvcConfig implements WebMvcConfigurer {  
    @Override  
    public void addInterceptors(InterceptorRegistry registry) {  
        registry.addInterceptor(new TestInterceptor())  
            .addPathPatterns("/hello");  
  
        /*  
        registry.addInterceptor(인터셉터객체)  
            .addPathPatterns("/") - 모든 Path 적용  
            .excludePathPatterns("/sample"); - /sample Path 에 대해서만 제외  
        */  
    }  
}
```

addInterceptor() : 등록할 인터셉터 설정

addPathPatterns() : 적용할 URL 패턴 설정

excludePathPatterns() : 인터셉터를 제외할 URL 패턴 등록

## [ 필터(Filter) vs 인터셉터(Interceptor) 차이 정리 및 요약 ]

| 대상                           | 필터(Filter)                                                                                                                                                       | 인터셉터(Interceptor)                                                                                                                            |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 관리되는 컨테이너                    | 서블릿 컨테이너                                                                                                                                                         | 스프링 컨테이너                                                                                                                                     |
| 스프링의 예외처리 여부                 | X                                                                                                                                                                | O                                                                                                                                            |
| Request/Response 객체 조작 가능 여부 | O                                                                                                                                                                | X                                                                                                                                            |
| 용도                           | <ul style="list-style-type: none"><li>• 공통된 보안 및 인증/인가 관련 작업</li><li>• 모든 요청에 대한 로깅 또는 감사</li><li>• 이미지/데이터 압축 및 문자열 인코딩</li><li>• Spring과 분리되어야 하는 기능</li></ul> | <ul style="list-style-type: none"><li>• 세부적인 보안 및 인증/인가 공통 작업</li><li>• API 호출에 대한 로깅 또는 감사</li><li>• Controller로 넘겨주는 정보(데이터)의 가공</li></ul> |

[ Spring MVC 의 전체적인 처리 흐름 ]

